Minor Project

Predictive Classification using Random Forest Ensembles

Predicting Whether Annual Income Exceeds 50K (Adult Census Dataset)

Name: Akshat Shankar Bidwai

1. Project Objective

The objective is to apply the Random Forest algorithm to a real-world classification problem: predicting whether a person earns more than 50K a year (class 1) or not (class 0) from census attributes such as age, education, occupation and hours worked. The project covers data cleaning, EDA, model training, hyper-parameter tuning and evaluation with professional metrics.

2. Dataset

The **Adult Census Income** dataset (`adult.data`, UCI / Kaggle) has 32,561 rows and 15 columns (6 numeric, 9 text). The target is **income**. The file has no header row, so column names were supplied manually. Dataset link: https://archive.ics.uci.edu/dataset/2/adult

3. Data Cleaning

- **Income column:** the target was stored as text (*object*). Spaces, trailing full stops and letter-case differences were removed so that every value is one of two clean labels, then mapped `<=50K` → `0` and `>50K` → `1` and converted from *object* to *int*. A check confirmed no value was left unmapped. In this file the only formatting problem found was the leading space before each label (e.g. " <=50K").
- **Missing values:** the file uses "?" instead of NaN (1,836 in workclass, 1,843 in occupation, 583 in native_country). These were converted to NaN and filled with the most frequent value (mode) of each column.
- **Duplicates:** 24 duplicate rows were removed, leaving 32,537 rows.
- **Dropped columns:** *education* (same information as the numeric *education_num*) and *fnlwgt* (census sampling weight, not a personal attribute).
- **Encoding:** the remaining text columns were converted to numbers with Label Encoding.

4. Exploratory Data Analysis

Class imbalance: about 75.9% of people earn <=50K and only 24.1% earn >50K. Because of this, accuracy alone is misleading (always predicting "<=50K" already gives about 76%), so Precision, Recall and F1-score for the >50K class are the main metrics.

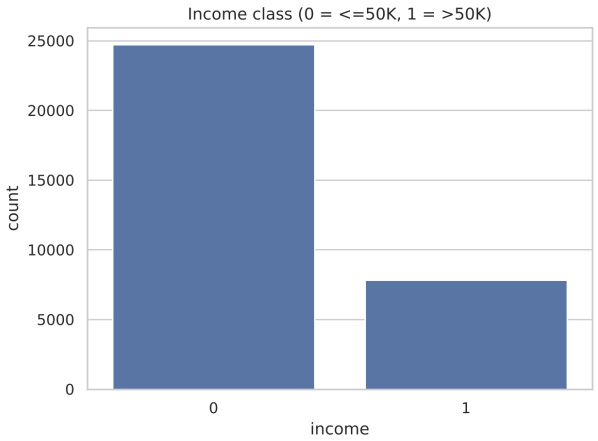


Figure 1: Income class distribution

Key observations from the plots:

- Higher income is more common among **married** people, **men**, and people with **more years of education**.
- People earning >50K tend to be older and to work more hours per week (Figure 3).
- Outliers exist (e.g. very large capital gains) but they are genuine values, and Random Forest is not sensitive to them, so they were kept.

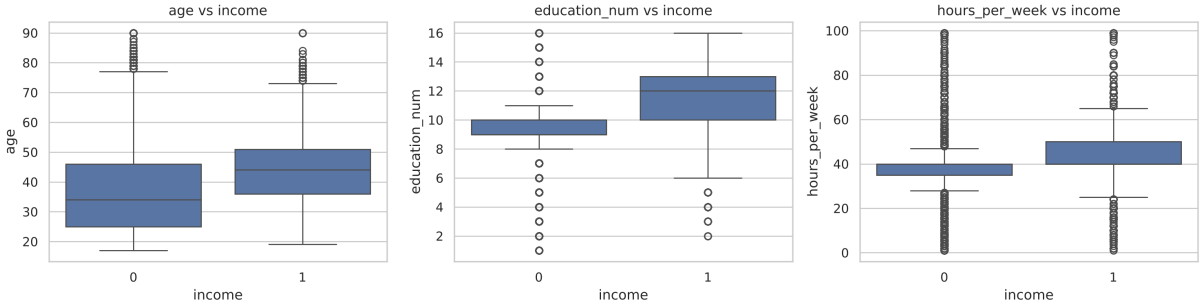


Figure 2: Age, education and weekly hours versus income

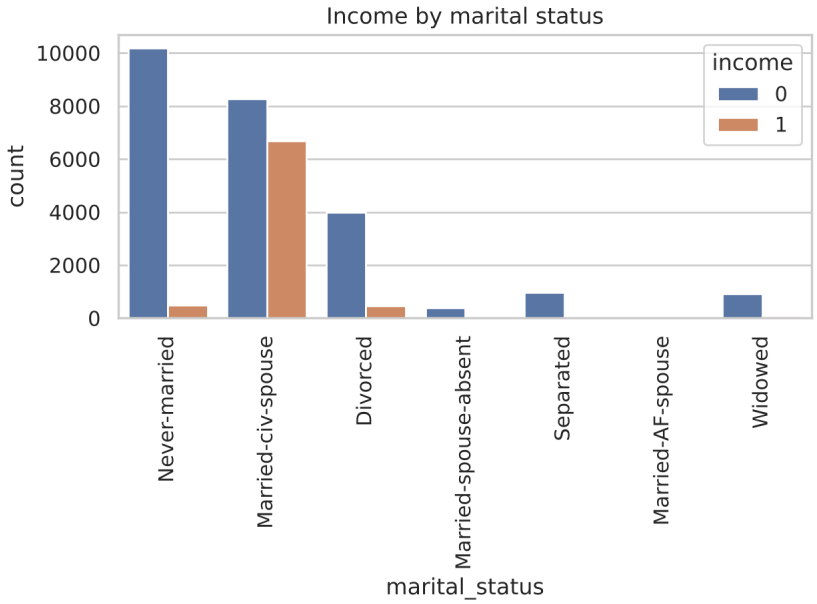


Figure 3: Income by marital status

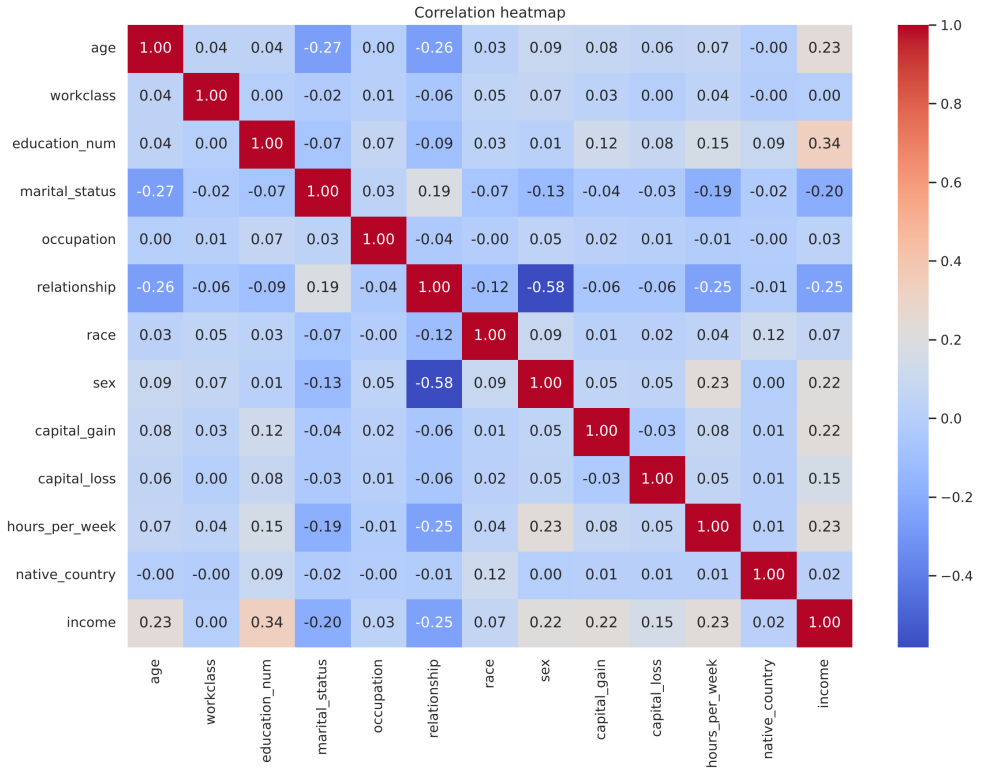


Figure 4: Correlation heatmap (after encoding)

The heatmap shows that income is most correlated with *education_num*, *age*, *hours_per_week* and *capital_gain* (positive) and with *marital_status* / *relationship* (negative, because of the way the labels are encoded). No pair of features is so strongly correlated that it needs to be removed.

5. Preprocessing and Model Development

- **Train/test split:** 80:20, stratified so both parts keep the same income ratio (26,029 training rows, 6,508 test rows).
- **SMOTE:** synthetic minority-class samples were added to the *training data only* to balance the classes. The test set stays untouched.
- **Feature scaling skipped:** Random Forest splits on thresholds rather than distances, so scaling does not change the result.
- **Baseline model:** RandomForestClassifier with 100 trees and default settings reached ~97% train accuracy but only ~83.5% test accuracy - clear **overfitting**.

Hyper-parameter tuning (GridSearchCV, 3-fold, scoring = F1)

Hyper-parameters were chosen to control overfitting. SMOTE was placed inside the cross-validation pipeline so that synthetic samples never leak into the validation fold.

Parameter	Values tried	Why it matters	Best
n_estimators	100, 200	More trees give more stable votes but take longer	200
max_depth	None, 10, 20	Limits how deep each tree grows (controls overfitting)	None
min_samples_leaf	1, 2, 4	Larger leaves give smoother, simpler trees	4

The best combination was 200 trees with no depth limit and **min_samples_leaf = 4**. The leaf-size constraint was the setting that mattered: it reduced the train accuracy from ~97% to 87.5% while test accuracy rose to 84.1%.

6. Evaluation (Tuned Model, Test Set)

Class	Precision	Recall	F1-score	Support
0 (<=50K)	0.94	0.85	0.89	4940
1 (>50K)	0.63	0.82	0.71	1568

Test accuracy = **84.1%**, ROC-AUC = **0.919**, F1 (>50K) = **0.71**. Compared with the untuned baseline (F1 0.68 for >50K), tuning improved F1 and recall for the minority class.

Confusion matrix

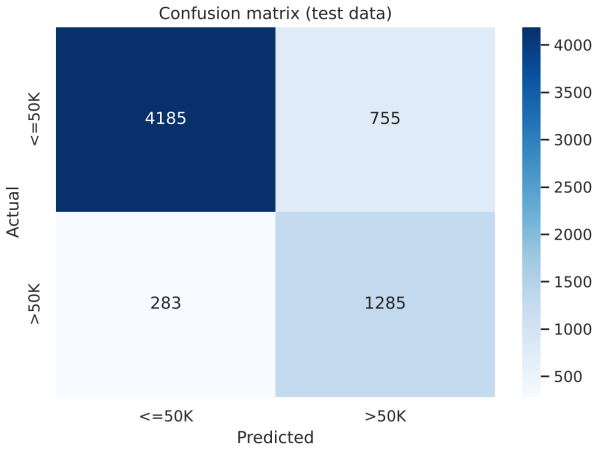


Figure 5: Confusion matrix on the test set

Out of 6,508 test people: **4185** true negatives, **755** false positives, **283** false negatives and **1285** true positives.

Precision vs. Recall trade-off

Recall (0.82) means the model finds about 82% of people who really earn >50K; only 283 were missed. **Precision (0.63)** means that when it predicts >50K it is right about 63% of the time; 755 people were wrongly flagged. SMOTE pushed the model towards recall, which is useful if the goal is to *not miss* high earners (e.g. identifying candidates for a premium product), but it costs precision. If wrongly flagging people were more costly, the decision threshold could be raised to favour precision. The F1-score (0.71) balances both and is the right summary here because the classes are imbalanced.

ROC curve

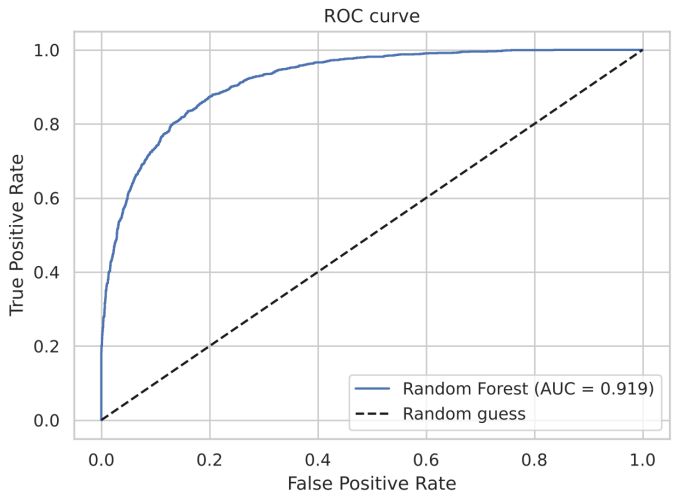


Figure 6: ROC curve

The AUC of **0.919** means that if we pick one high earner and one non-high earner at random, the model ranks the high earner higher about 92% of the time - far better than random guessing (0.5).

Feature importance

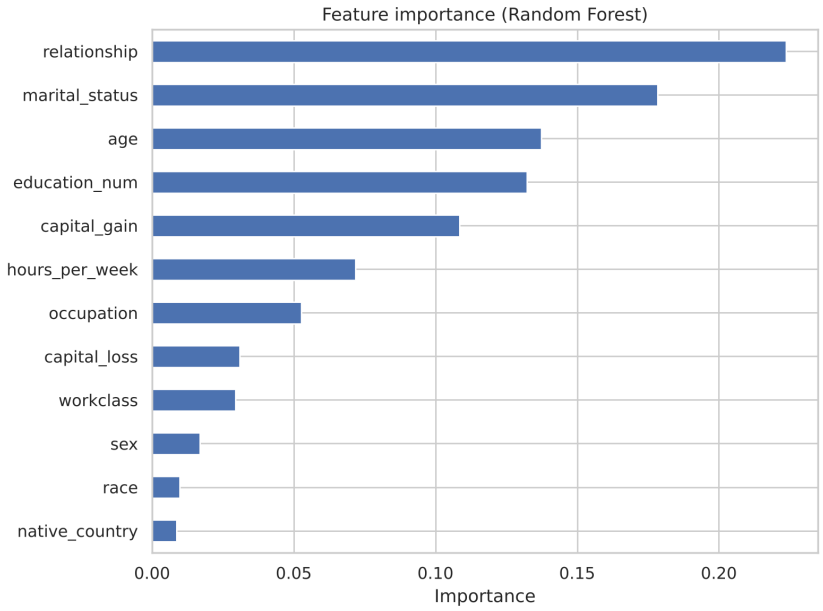


Figure 7: Feature importance

The top five features are *relationship* (0.22), *marital_status* (0.18), *age* (0.14), *education_num* (0.13), *capital_gain* (0.11). This matches intuition: income depends mainly on age/experience, family situation, education and working hours, while *race*, *sex* and *native_country* contribute little.

7. Why Did We Get These Results?

Reporting numbers is not enough, so each result was tested to find the reason behind it (Section 10 of the notebook).

7.1 Why is recall high (0.82) but precision lower (0.63)?

The same tuned model was trained with and without SMOTE:

Model	Precision (>50K)	Recall (>50K)	F1 (>50K)
Without SMOTE	0.79	0.64	0.71
With SMOTE	0.63	0.82	0.71

SMOTE makes the training data 50/50, so the model "believes" high earners are much more common than the real 24%. It therefore predicts >50K more often: it catches more real high earners (**recall rises from 0.64 to 0.82**) but also wrongly flags more people (**precision falls from 0.79 to 0.63**). The F1-score is almost unchanged, so SMOTE *shifts* the balance between precision and recall rather than making the model better overall. Without SMOTE the model would be cautious: when it says >50K it is usually right, but it misses about 36% of real high earners. Which version is better depends on the cost of each mistake.

7.2 Why does the model miss some high earners (283 false negatives)?

Caught high earners had an average *capital_gain* of 5,208 and 12.0 years of education, but the missed ones averaged only 140 and 10.1 years. The model leans heavily on these signals, so a high earner without large capital gains or a higher education looks just like an ordinary earner and is missed.

7.3 Why does it wrongly flag some people (755 false positives)?

Group (test set)	Avg age	Avg education yrs	Avg hours/week
Caught high earners (TP)	44.8	12.0	45.5
Missed high earners (FN)	42.4	10.1	44.2
Wrongly flagged low earners (FP)	43.1	10.9	44.4
Correct low earners (TN)	35.9	9.3	38.0

The wrongly flagged people are older (43.1 vs 35.9 years), more educated (10.9 vs 9.3 years) and work longer hours (44.4 vs 38.0) than the typical low earner. They genuinely *look like* high earners on the available features, so some of these errors cannot be avoided without extra information (e.g. job title or industry).

7.4 Why did the default model overfit, and why did tuning help?

Fully grown trees keep splitting until each leaf holds one training row, so they memorise individual (and synthetic SMOTE) samples, including noise. This gave ~97% train accuracy but only ~83.5% test accuracy. Setting *min_samples_leaf = 4* forces every leaf to represent several people, which produces smoother rules that generalise: the train-test gap shrank (87.5% vs 84.1%) and the >50K recall and F1 improved.

7.5 Why is accuracy (84.1%) not the whole story?

About 76% of people earn <=50K, so a model that always answers "<=50K" would already score ~76%. This is why the report focuses on precision, recall and F1 of the >50K class, and why the <=50K class has high scores (F1 0.89) simply because it is the majority class and easier to learn.

8. Conclusion

Strengths

- Good ranking ability (AUC 0.92) and 84.1% test accuracy, with high recall (0.82) on the minority >50K class.
- Handles mixed numeric/encoded features without scaling and is robust to outliers; feature importance makes the result easy to explain.
- Tuning with *min_samples_leaf* clearly reduced overfitting (train 97% → 87%).

Weaknesses

- Precision for >50K is only 0.63 - about 755 people were falsely labelled as high earners.
- Label Encoding gives categories an artificial order; Random Forest copes with this, but One-Hot Encoding may be cleaner.
- The small grid (18 combinations) may not have found the best settings; "?" values were filled with the mode, which can add bias.

Possible improvements

- Try Gradient Boosting / XGBoost / LightGBM, which often do better on tabular data.
- Tune more parameters (*max_features*, *class_weight*) and adjust the probability threshold to balance precision and recall.
- Use One-Hot Encoding and compare with SMOTE vs. no SMOTE.